Search-Eliminating Computation — The Method of Pressure-Field Compression

A Non-Search Approach to Discrete Constraint Satisfaction Problems: Proposal and Circuit Implementation

目次

Abstract

Many computational methods for discrete constraint satisfaction problems place at their core a search that traverses a space of candidate solutions. This paper proposes a computational mode that differs from this. That is: a non-search paradigm that generates multiple local pressures from the problem state, projects them onto a single field, and determines only when the pressure difference becomes decisive.

The central idea of this paradigm lies in realizing determination not as "choice" but as "compression." When multiple pressure events are projected onto a single field (the A-field), a forced determination appears as a sharp pressure difference, and when that pressure difference is decisive, determination fixes at once. On the other hand, in an ambiguous situation where multiple possibilities contend, the pressure difference stays gentle and determination does not occur. In an ambiguous situation, this paradigm makes no choice and suspends determination. Contradiction is sent, separately from determination candidates, to a contradiction sink.

We implemented this mode at both the program level and the hardware-circuit level. At the program level, we verified the validity of the framework over a body of program-level problems (the figures in this section are by program-level measurement, a line of evidence separate from the circuit-level waveform verification described in Chapter 9). At the circuit level, we implemented it as 20 hardware-description modules, equipping it with static and waveform inspection ports. In an environment where waveform simulation is available, we verified by waveform the flow of determination through pressure sources, the pressure bus, and the collapse cell (in an environment without a simulator, waveform execution is suspended). Furthermore, we confirmed that the output of this paradigm agrees with an independently prepared reference for checking, and quantitatively observed the continuity of the pressure gradient.

This paradigm is characterized by realizing determination as "pressure-field compression," by having none of search, backtracking, stochastic retry, or flight into existing solvers, and by clearly distinguishing forcing, ambiguity, and contradiction. This paper shows that this holds at the level of implementation and waveform verification, in both software and hardware.

1. Introduction

1.1 Background

Computational methods for discrete constraint satisfaction problems and discrete NP-hard problems are, for the most part, founded on search. Conflict-driven learning in satisfiability, backtracking in constraint programming, and stochastic local search with variable flipping in hardware solvers all share the trait of traversing a space of candidate solutions by some policy. While these handle a broad range of problems, as long as they presuppose search, their worst-case complexity can be exponential in problem size.

What this paper asks lies in a different direction. That is: rather than searching for the solution, can we let the determination that the problem forces surface as a physical pressure difference, and fix determination only when that pressure difference is decisive — thereby making search itself unnecessary?

1.2 Idea

In certain constraint satisfaction problems, there are consequences uniquely fixed by the initially given constraints. Implication from a unit clause, and exclusion by a capacity constraint, are typical, and none involve "choice." These forced determinations can be expressed as multiple local pressures, and superimposing them onto a single field produces a sharp pressure difference in the direction of forcing.

What matters here is that the sharpness of the pressure difference reflects how strongly the determination is forced. If forcing is decisive, the pressure difference is sharp and determination fixes at once. If multiple possibilities contend, the pressure difference stays gentle and determination does not occur. This paradigm places at its center the idea of "physically deciding the possibility of determination by the sharpness of the pressure difference."

1.3 Contributions

The contributions of this paper are fourfold. First, we formalize the design philosophy of realizing determination as pressure-field compression, and show the division of roles between Φ (the computation that carries out compression) and λ (the computation that carries out inspection) that supports it (Chapters 2–4). Second, we implement this philosophy at the program level as a unified framework applicable to 13 problem families (Chapter 5). Third, we quantitatively distinguish three behaviors — compression of forced structure, non-choice of ambiguous structure, and sinking of contradiction — and verify them by comparison against an independently prepared reference for checking and by the continuity of the pressure gradient (Chapters 6 and 7). Fourth, we implement this mode as a hardware description, and verify, in an environment where waveform simulation is available, its holding at the level of circuit description (Chapters 8 and 9).

2. Design Philosophy

2.1 Do Not Build a Third Solver

At the root of this paradigm's design philosophy lies a discipline: "do not build a third solver."

An implementation of constraint satisfaction often, when it hits a wall, flees into existing computational methods. Such operations include introducing search as an aid, trying other candidates by backtracking, retrying stochastically, or taking in the machinery of existing solvers. These raise apparent solving power, but at that moment the paradigm ceases to be "computation that does not search."

This paradigm forbids these at the design stage. When determination does not fix as pressure-field compression, the cause lies in how the pressure field is built — that is, in the quality of the projection from problem to field. We call this the "quality debt of A." Even with debt, the system does not rescue itself by branching, backtracking, retrying, or importing existing computational methods. It leaves debt as debt, and honestly shows that it cannot determine.

This discipline means that this paradigm is a framework that keeps two equal computations (Φ, which carries out compression, and λ, which carries out inspection) within the same runtime — not a third solver with a fixed order of execution. This boundary is also fixed explicitly in implementation: taking in search, backtracking, recursion, or beam search as auxiliary means, and filling gaps with the computational mode of an existing solver, are all described as prohibited items.

2.2 Realize Determination as Compression

From the discipline above follows the central idea of this paradigm. That is: realize determination not as "choice" but as "compression."

Ordinary computation "chooses one from several candidates, and backtracks if it errs." Because there is choice and backtracking, this becomes search. Against this, this paradigm projects multiple local pressures onto a single field, and fixes determination only when the pressure difference of that field is decisive. If the pressure difference is not decisive, determination does not occur. Rather than choosing, it fixes naturally when pressure converges to a single point.

Here the key is how to treat "contending situations." This paradigm makes no choice in such situations. Multiple possibilities are held in contention, as a gentle pressure field. If new forcing sharpens the pressure difference, determination resumes; if not, determination does not occur. As long as it forbids choice, this paradigm is in principle incapable of search.

2.3 Design Principles

To secure the philosophy above, this paradigm explicitly prescribes prohibitions. It has none of: branching, backtracking, recursion, guessing hidden facts, importing the core of existing solvers, running rescue plugins, undo or restart, or forcibly fixing values. Each of these amounts to either "search" or "flight into existing computational methods." By structurally excluding these, we guarantee that this paradigm is neither search nor a rephrasing of an existing solver.

3. The Pressure Field and the A-Field

The core of this paradigm is a mechanism that expresses the determination the problem forces as pressure, and projects them onto a single field. Whatever the problem, through this projection, determination surfaces as a pressure difference.

3.1 Pressure Events and Projection

Each local structure of the problem is expressed as a pressure event that promotes determination. The pressure of a unit clause in satisfiability, and the pressure of remaining capacity and value in knapsack, are examples. These pressure events exist multiply at once, each having a direction and strength of determination.

This paradigm projects these multiple pressure events onto a single field — the A-field. In the projected field, pressures superimpose in each candidate's determination direction, forming a sharp pressure difference in the direction where forcing is decisive, and a gentle one in the contending direction. Determination fixes, only when the pressure difference of this field is decisive, as a single decisive determination event.

3.2 The Pressure Difference Decides the Possibility of Determination

The sharpness of this pressure difference decides the possibility of determination. In situations where the pressure difference is decisively sharp, a forced determination fixes uniquely and requires no search. In situations where the pressure difference stays gentle, multiple possibilities contend and determination does not occur. That is, this paradigm physically expresses the boundary of "can the problem determine, or stay in contention" as a pressure difference of the field.

The quality of determination depends on how the pressure field is built. If forcing is correctly expressed as pressure, the pressure difference appears sharp in the direction of forcing. If the expression is insufficient, the pressure difference stays gentle even where determination should occur. In that case, it is the aforementioned "quality debt of A," and this paradigm does not compensate for it by search.

3.3 Continuity of the Pressure Field

We confirmed quantitatively, across multiple problem families, that the pressure field has not a mere binary decision but a continuous gradient. Measuring the pressure gradient for the satisfiability, knapsack, and subset-sum families, the ratio of pressure difference to pressure spread was in all cases extremely small, and a region was observed where the pressure field is continuous and gently distributed. This continuous region corresponds to situations where determination is not decisive — situations where contention is held. The pressure field expresses, as a continuous quantity, both determination by a sharp pressure difference and non-choice by a gentle one.

4. The Composition of κ and the Division of Roles Between Φ and λ

This paradigm forms part of a larger computational system. That system is composed as the division of roles between two computations — the computation that carries out

compression (Φ-computing), and the computation that carries out its inspection (λ-computing). This is the greatest feature distinguishing this paradigm from existing computational modes.

4.1 The Composition of Two Computations

We call the computational system to which this paradigm belongs κ-computing. κ-computing is composed of the computation that realizes determination as compression (Φ), the computation that inspects its run (λ), and the computation that generates hypotheses ($\mathcal{M}$). Of these, hypothesis generation ($\mathcal{M}$) belongs only to a higher level of abstraction and is not treated in this paper. Φ and λ both exist at the program level and cooperate as separate computations.

What matters here is that κ-computing is not a third solver with a fixed order of execution. κ-computing is the very boundary that keeps two equal computations (Φ and λ) within the same runtime, ensuring that neither strays into search or existing methods. This boundary is, in implementation, fixed as a dedicated invariant.

4.2 Φ: Compression and Single-Trajectory Motion

What Φ carries out is fixing determination as compression. Φ stays within three constraints: building the structure of the solution, not searching, and compressing computation instead of widening a tree of candidates.

At the program level, Φ projects multiple pressure events onto a single A-field, and advances only with the single determination that became decisive in pressure difference. If motion is poor, it is the quality debt of A, and Φ does not rescue itself by branching, retrying, or introducing existing-solver machinery. Φ does not use the weakness of A as a pretext to take in another computational mode. It advances a single trajectory, and if determination does not fix, it honestly shows that it does not fix.

4.3 λ: Inspection, the Origin of Learning, and Human Intervention

λ is a separate computation that inspects Φ's run. λ stays within three constraints: inspection, the origin of learning, and human intervention.

λ reads the traces Φ has emitted, names the shape of situations where determination has halted, and develops a single observed source into multiple origins of learning for a human to address later. What matters is that λ does all of these strictly by reading. λ does not change Φ, does not choose determination events, does not repair automatically, and does not close its own loop.

This read-only nature is a design choice of this paradigm. λ opens learning only up to "origins." It does not turn knowledge obtained from observation into automatic correction commands during the run, treat it as an immediate repair policy, or close it as a self-completed learning result. Completing learning, and thereafter changing the mode of determination, is the role of a human outside the run. That λ does not feed results back into the run is fixed as an invariant in implementation.

4.4 Connection, Not Mixing

That Φ and λ are separate computations raises the question of how to connect them. This paradigm does not blend them together but connects them at limited points.

Legitimate connection is limited to two points. First, the connection from run to inspection — after Φ fixes a determination, or after determination halts, λ reads its trace. Second, the connection from inspection to human — the origin of learning that λ has opened is received by a human outside the run.

Against this, there are mixings that are forbidden: turning λ's observation into automatic commands during the run, treating the origin of learning as an immediate repair policy, bringing higher-level machinery in during the run, and filling gaps with the existing computational mode of another solver. Each of these breaks the separation of compression and inspection, and drags this paradigm back toward search or existing solvers. The two are kept as separate computations and connected only at prescribed points.

5. Program-Level Implementation

5.1 Unified Framework and Problem Families

We implemented this paradigm at the program level and composed it as a unified framework for discrete NP-hard problems. The targets span 13 problem families, including clique, binary equations, satisfiability, tour decision, Sudoku, graph coloring, knapsack, subset sum, set cover, vertex cover, independent set, and max cut.

These problem families are all implemented as pressure sources. The local structure of each problem generates pressure events that promote determination. What is prepared per problem is only the pressure generation specific to that problem — enumerating valid moves, evaluating each as pressure, and applying the chosen one; projection onto the pressure field and the core mechanism of determination are common to all problems. A problem is implemented not as an independent solver but as a source feeding pressure into the common motion loop.

5.2 The Single-Trajectory Motion Loop

The core of this framework is a single-trajectory motion loop. The loop gathers valid moves from pressure sources, evaluates each move as a pressure event, projects multiple pressure events onto a single A-field, applies the selected event, writes a memory of the local terrain, and emits a trace.

This loop does not branch, does not backtrack, does not recurse, does not guess hidden facts, does not take in the core of existing solvers, does not run rescue plugins, and does not use undo or restart. Even if determination does not fix, the loop does not transform into search. The memory of local terrain stays a light hint for determination, and never chooses or rejects the next move. Whether memory begins to dominate determination is monitored by read-only inspection.

5.3 Compression of Forcing, and Honest Halting

We verified the validity of this framework over a body of program-level problems (the figures in this section are by program-level measurement, a line of evidence separate from the circuit-level waveform verification described in Chapter 9). In the core benchmark, all 44 cases succeeded. On the other hand, in a load benchmark deliberately made hard, in the initial observation window all 25 cases halted without reaching determination (henceforth we call this halt, where determination does not fix, a non-determination halt). These are situations where the pressure difference of forcing does not fix decisively and determination did not occur.

What matters here is that these halts are not defects. The non-determination halts of the load benchmark are situations where, because this paradigm does not search, it cannot determine, and it displays that fact without falsifying. The remaining halts are not "unresolved homework" but are held as evidence for later analysis. For this paradigm, a halt that honestly shows non-determination is more trustworthy than a chance determination.

Furthermore, we confirmed the purity of solving from multiple angles. Changing the order of processing pressure events, the conclusions agreed (order-independence); repeating the same input, the conclusions were completely identical (determinism). Every solution judged as determined was verified valid against the problem's definition (there was no false determination). Running heterogeneous problems in succession, each conclusion agreed with the isolated run (no state leakage). And each determination could be traced, with its source, to which pressure forced it (traceability). These show that there is no hidden guessing or state contamination in the process by which this paradigm fixes determination as compression.

6. The Three Behaviors of Forcing, Ambiguity, and Contradiction

6.1 Distinguishing the Three Behaviors

This paradigm shows three behaviors according to the local structure of the problem, and clearly distinguishes them. Forced structure is compressed and determined. Ambiguous structure is held as non-choice. Contradictory structure is sunk. This distinction is the core of this paradigm's ability to "show that it cannot determine when it cannot," without fleeing into search.

6.2 Forcing: Determination by Compression

In a forced situation, the pressure difference becomes decisively sharp and determination fixes at once. A unit clause in satisfiability, the constraints of remaining capacity and value in knapsack, the only possible color in graph coloring, the only coverer in set cover, the only possible slot in scheduling, and so on, produce this sharp pressure difference. In these situations, this paradigm fixes determination without any search or backtracking.

6.3 Ambiguity: Holding by Non-Choice

In a situation where multiple possibilities contend, the pressure difference stays gentle. A knapsack with equivalent items lined up, graph coloring with multiple possible colors, set cover with overlapping coverers, scheduling with multiple possible slots, and so on, produce this gentle pressure field. In these situations, this paradigm makes no choice and holds the contention as non-choice. It does not choose one by guesswork. This is not a defect but a design behavior — where determination is not forced, not determining is correct.

However, it became clear in measurement that the purity of this non-choice depends on the setting (the dial) of how far determination is permitted. In the loosest default setting, part of the contending situations, or part of the inputs lacking a forcing origin, may be determined rather than held. On the other hand, in the strict-side setting, both origin-lacking inputs and symmetric contention were held as non-choice, and problems where forcing reaches were determined as before. In a setting intermediate between the two, a behavior was obtained that determined all problems where forcing reaches while holding contention. This means that the design of "being able to choose the degree of involvement with Φ," described in Chapter 4, also acts on the strictness of non-choice. To keep non-choice strict requires a corresponding setting, and the default setting is loose in that respect — we record this fact without hiding it.

6.4 Contradiction: Sinking

In a situation where the problem state structurally cannot hold, the pressure field collapses and contradiction is sunk. The cases where the initial state is already contradictory, where a determination move crushes the remaining space, and where the pressure of forcing loses its way under tightening, produce this collapse. The sinking of contradiction, too, involves no search or backtracking. This paradigm does not "search trying to solve" a contradiction, but sinks it as a structure that cannot hold.

These three behaviors are a natural consequence of the determination physics being pressure-field compression. A sharp pressure difference produces compression (forcing), a gentle one produces non-choice (ambiguity), and a collapsed pressure field produces sinking (contradiction). None pass through choice or search; all emerge from the state of the pressure field itself.

7. Verification

We verified the validity of this paradigm from multiple sides: the core benchmark, the load benchmark, comparison against an independently prepared reference for checking, and the continuity of the pressure gradient.

7.1 Core Benchmark and Load Benchmark

In the core benchmark, all 44 cases succeeded. This shows that for problems with forced structure, the pressure difference fixed decisively and determined without search. In a benchmark of 100 problems made hard, all cases completed processing.

All 25 cases of the load benchmark, in the initial observation window, became non-determination halts. These are situations where determination does not fix, classified by six boundary labels — contradiction of the initial state, absence of a valid pressure surface before moving, collapse of the space after a choice, closure of the observation window, disappearance of space by tightening, and flattening of the pressure field. These labels are neither corrections, policies, nor retry commands, but boundary descriptions naming the shape of situations where determination has halted.

7.2 Comparison Against an Independent Reference for Checking

We verified that the determinations of this paradigm are valid by comparison against an independently prepared reference for checking. For the satisfiability, subset-sum, knapsack, and Sudoku families, comparing the determinations this paradigm fixed by compression against a built-in independent reference for checking, all 4 cases agreed. The inputs used for checking are written out in a standard format, reproducible independently by a third party. This is a confirmation that the determinations this paradigm fixed do not contradict an independently prepared reference for checking. Note that this is not a head-to-head superiority comparison against an external solver under identical conditions.

7.3 Continuity of the Pressure Gradient

We quantitatively observed that the pressure field has a continuous gradient. Measuring the pressure gradient for the satisfiability, knapsack, and subset-sum families, in all cases the ratio of pressure difference to pressure spread was extremely small, and a region was confirmed where the pressure field is continuous and gently distributed. This continuous region corresponds to situations where determination is not decisive — situations where contention is held as non-choice. It shows that determination by a sharp pressure difference and non-choice by a gentle one are expressed as a single continuous quantity.

7.4 Fixing of Purity

That this paradigm does not stray into search or existing methods is continuously verified as an implementation invariant. That Φ, which carries out compression, contains no search, backtracking, or import of existing computational methods; that λ, which carries out inspection, does not feed results back into the run; and that the boundary binding the two does not build a third solver, are each fixed as independent checks. That these checks keep holding is the indicator that this paradigm keeps its purity.

7.5 The Relation Between Strictness of Determination and Coverage

Changing the setting of how far determination is permitted changes how many problems are determined and how many are left as non-choice. For the same 12 satisfiability problems, the default setting determined all 12. The strict-side setting determined 8 and held the remaining 4 as non-choice. The intermediate setting determined all 12 while holding the contention that should be held.

This shows that determination coverage and the strictness of non-choice are in a trade-off relation, and that one can choose their balance by setting. Made strict, in exchange for honestly holding rather than determining problems where forcing is not decisive, the problems determined decrease. Made loose, coverage increases but room arises to determine contention. Which is desirable depends on the use — for uses that most prioritize the legitimacy of determination, the strict side; for uses seeking coverage, the intermediate. Having this choice itself is the design flexibility of this paradigm.

8. Circuit Level: Physicalizing the Pressure Field

This paradigm is implemented not only at the program level but also as a hardware circuit. The circuit level realizes pressure-field compression as a hardware description, showing that this paradigm's mode of determination is not peculiar to software but holds at the level of circuit description and waveform simulation. Note that verification at the physical layer (process-level element design, physical placement, circuit simulation, silicon measurement) is out of scope for this paper and is future work. Also, λ-computing, which carries out inspection, is not an object of circuit-level measurement. What this chapter treats is the circuit realization of Φ, which carries out compression; λ's inspection, origin of learning, and human intervention are program-level roles as described in Chapter 4, and lie on a different layer from the circuit waveform evidence.

8.1 Oracle Candidates on the Circuit Side

At the circuit level, the structure carrying determination is composed as three pressure dimensions, A = (AH, AJ, AC). AH is pressure given from outside, AJ is the pressure of parallel interference, and AC is the pressure of concurrent local determination. The pressure that the local structure of the problem generates is sorted into these dimensions.

Note that these symbols refer to the pressure dimensions on the circuit side of this paper. The same symbols appear in a separate line of paradigm treated in a sister paper, but there the meaning (distinction of information sources) is a different concept.

8.2 Routing of Pressure

The targets implemented at the circuit level are five problem families: satisfiability, knapsack, graph coloring, set cover, and scheduling (of the thirteen program-level families, these five are the ones whose circuit realization was advanced). For each problem family, which dimension the pressure is sorted into is fixed. The pressure of satisfiability goes to AJ (parallel interference); the pressure of knapsack, graph coloring, set cover, and scheduling goes to AC (concurrent local determination). In satisfiability, the pressure of a unit clause; in knapsack, the pressure of remaining capacity and value and the limiting pressure by fractional relaxation, form the field through their respective dimensions.

8.3 The Pressure Bus and the Collapse Cell

The pressure of each dimension is integrated by the pressure bus into a single field. The pressure bus superimposes the external, interference, local-determination, residual, and cross pressures non-linearly. The integrated field is passed to the collapse cell. The collapse cell, equipped with a threshold and hysteresis, collapses into determination only when the pressure difference of the field is decisive. If the pressure difference is not decisive, collapse does not occur and determination is suspended. Contradiction is sunk at the collapse cell.

This circuit has neither a central controller nor a max-selector. Determination fixes naturally at the collapse cell when the pressure difference of the field formed by the pressure bus becomes decisive. The circuit is implemented as 20 hardware-description modules, including pressure sources, the pressure bus, the collapse cell, residual memory, cross pressure, and the top wrapper of each problem.

9. Waveform Verification

To verify that the circuit operates as intended, we conducted waveform simulation. Verification centered on two top paths — the AJ path of satisfiability and the AC path of knapsack.

9.1 Composition of Verification

Waveform verification was conducted using two simulators. For the pressure cell alone, the residue/cross/bus/collapse of satisfiability, the two top paths, and the oracle-boundary circuit that shows the boundary of forcing, ambiguity, and contradiction in a single waveform, we recorded waveforms and checked final states. In an environment where a waveform simulator is available, in aggregate all 6 waveform records succeeded in checking, and all 26 final-state expectations agreed. In an environment without a simulator, although the ports for static and waveform inspection are present, the execution of waveforms is suspended. The waveform results of this chapter were obtained in an environment with a simulator installed.

9.2 The AJ Path of Satisfiability

The waveform of the top path of satisfiability confirmed the execution of the actual top circuit, that the pressure of satisfiability is sorted into the AJ dimension, the formation of residual and cross pressure, the separation of the signed field, the collapse into determination, and that no contradiction arose. The flow in which pressure forms the field through AJ and collapses into determination when that field's separation becomes decisive was observed as a waveform.

9.3 The AC Path of Knapsack

The waveform of the top path of knapsack confirmed the execution of the actual top circuit, that the pressure of remaining capacity and value and the limiting pressure by fractional relaxation are sorted into the AC dimension, the separation of the include/exclude field of items, the fixing of determined items, and that no contradiction arose. The flow in which the pressure of a problem other than satisfiability (non-SAT) forms the field through AC and reaches determination was observed as a waveform.

9.4 The Boundary Waveform of Forcing, Ambiguity, and Contradiction

We confirmed, in a single waveform, that the three behaviors described in Chapter 6 are distinguished on one circuit. This circuit gives three phases in turn to the same oracle boundary.

In the contradiction phase, when a contradiction input was given under strong pressure, the signal indicating contradiction rose, the response became a contradiction state, and determination of the target was suspended. In the ambiguous phase, when even pressure in two directions (contention of equal magnitude) was given, no contradiction arose, the response stayed a free state, and the target was not determined — even pressure was not determined by guesswork but held as non-choice. In the forcing phase, when a decisive pressure difference was given, the target was determined to the expected value and no contradiction arose.

That these three phases were observed in a single waveform shows that the distinction of forcing, ambiguity, and contradiction arises not from a software case-split but from the physical condition of the circuit — whether the pressure difference crosses the collapse threshold and the hysteresis boundary. In particular, that the phase where even pressure is held as non-choice was confirmed by waveform is backing for this paradigm's discipline of "not filling contention by guesswork" at the circuit level.

9.5 Significance and Limits of Verification

These waveform verifications show that this paradigm's mode of determination holds at the level of circuit description and waveform simulation. Pressure-field compression runs not as a software abstraction but as a signal on the circuit.

However, let us state the limits honestly. The current circuit implementation stays at the hardware-description and reference-model stage. Process-level element design,

physical placement and routing, post-synthesis timing, silicon area, and measured power are all untouched. Waveform verification is completed for the top paths of satisfiability and knapsack and the boundary circuit; the top-level waveforms of other problem families, multi-valued determination beyond binary, and the simultaneous injection of multiple top-level pressure origins are not yet verified by waveform. These remain as items executable when the environment is in place.

There was one event worth recording in the process of verification. The waveform verification of the boundary circuit completed normally and passed checking on one simulator, but on the other simulator the process terminated abnormally due to the handling of a multidimensional structure. This is a matter of simulator support, not that the behavior of the circuit failed. We also record that the same circuit correctly generates a waveform and passes checking on the other simulator. What this chapter shows is that the mode of determination holds at the level of circuit description and waveform simulation, not its performance on silicon.

10. Discussion

10.1 Where the Value of This Paradigm Lies

The value of this paradigm lies neither in speed nor in coverage as a general-purpose solver. It lies in three things. First, the computational mode itself: realizing determination as pressure-field compression, and fixing forcing without choice or backtracking. Second, clearly distinguishing forcing, ambiguity, and contradiction as states of the pressure field, and honestly suspending determination in situations it cannot determine. Third, that this mode holds in both software and hardware, and can be implemented as a physical circuit design principle.

10.2 Structural Legitimacy of the Output

The second point matters for uses where reliability is at stake. This paradigm fixes determination by the sharpness of the pressure difference, and in contending situations holds without choosing. Thereby, the determinations this paradigm fixes are limited to those where pressure converged to a single point, not chosen by guesswork. For problems it cannot determine, it honestly suspends rather than returning a wrong determination. Comparison against an independently prepared reference for checking backs that the fixed determinations are correct. There is, here, a design choice not to sacrifice legitimacy in exchange for solving power.

10.3 Positioning Relative to Existing Hardware Solvers

Among hardware solvers specialized for satisfiability, there are those worth referencing in their metastability and the correspondence of clauses and variables. However, they differ in principle from this paradigm in using stochastic local search and variable flipping. This paradigm does not adopt these mechanisms. What is stated here is a reference comparison and positioning of design principles, not a superiority comparison of performance. A head-to-head comparison of the two requires synthesis

and silicon-level evaluation under identical conditions and benchmarks, and this is future work. What this paper claims at present is limited to this: a circuit mode that determines forcing without using search or variable flipping holds at the level of circuit description and waveform simulation.

10.4 On the Idea of Physical Determination of Forcing

The problem families this paradigm treats — satisfiability, graph coloring, tour decision, clique, knapsack, and so on — are all problem groups deemed hard in complexity theory. Against these, this paradigm consistently uses the fact that when a forced determination can be fixed as a physical pressure difference, search becomes unnecessary. That this boundary of "can determine / stays in contention" is itself physically expressed as a pressure difference of the field offers one view on where the difficulty of computation resides. How far its theoretical implications can be generalized is a problem requiring broader consideration beyond the scope of this paper.

11. Limits and Future Work

11.1 Limits

The determination of this paradigm is limited in range to situations where forcing fixes decisively as a pressure difference. In ambiguous situations where the pressure difference stays gentle, determination does not occur, and this paradigm holds them as non-choice. This is a design behavior, but uses requiring determination need combination with other means.

The purity of non-choice has a limit of dependence on the setting. In measurement, in the loosest default setting, cases were observed where part of the inputs lacking a forcing origin, or part of the symmetrically contending inputs, were determined rather than held. In the strict-side and intermediate settings these were held as non-choice, but that the default setting is loose in the purity of non-choice is stated explicitly as a present limit. To guarantee non-choice strictly requires the choice of setting, or a review of the default setting itself.

The circuit implementation stays at the hardware-description and reference-model stage; process-level element design, physical placement, post-synthesis timing, silicon area, and measured power are untouched. Waveform verification stays at the top paths of satisfiability and knapsack and the boundary circuit; the top-level waveforms of other problem families, multi-valued determination, and the simultaneous injection of multiple origins, as well as a comparison against a larger external solver, remain as items executable when the environment is in place.

This paradigm is not a general-purpose method that solves all hard problems in polynomial time. Nor does this paper claim a proof regarding the fundamental problem of complexity theory. What this paper shows is a concrete implementation that determines forcing as compression, without using search.

11.2 Future Work

There are three directions. First, to characterize more precisely, per problem family, the quality of the projection of the pressure field — the ability to correctly express the forcing that should be determined as a sharp pressure difference. Second, to advance the circuit implementation to synthesis, place-and-route, and the silicon level, clarifying its physical performance as dedicated hardware. Third, integration with another, complementary line of approach. If the ambiguous situations this paradigm holds as contention can be fixed from a different angle by another mode of determination, combining the two could greatly broaden the determinable problems. In particular, a sister paradigm that treats the same problem group with a different mode of determination — structural propagation — is a promising partner, in that it could determine, by a different route, situations where this paradigm's pressure field does not fix decisively. This integration is best done in a form that connects the two while keeping their boundary, after each mode is independently established; we treat it in a separate paper.

Appendix A: Reproduction Procedure

The program-level implementation of this paradigm is composed of a common motion loop and per-problem pressure sources. Solving each problem is carried out by giving the motion loop the pressure generation specific to that problem (enumeration of valid moves, evaluation as pressure, application). Verification is by the procedures of the core benchmark, the load benchmark, comparison against a reference for checking, and pressure-gradient measurement. Circuit-level verification is by the procedure of constructing, on a simulator, a circuit of pressure sources, the pressure bus, and the collapse cell, recording the waveform of the top path, and checking the final state. The tools and input formats needed for reproduction are provided as a reproduction guide and written-out input files.

Appendix B: The Routing Contract of Pressure

Problem family	Pressure dimension	Rule of compression	Behavior when ambiguous
Satisfiability	AJ	Pressure of a unit clause	Unresolved inputs held as non-choice
Knapsack	AC	Remaining capacity/value and limit by fractional relaxation	Equivalent items held as non-choice
Graph coloring	AC	The only possible color	Multiple possible colors held as non-choice
Set cover	AC	The only coverer	Overlapping coverers held as non-choice
Scheduling	AC	The only possible slot	Multiple possible slots held as non-

			choice

Appendix C: Key Terms

κ-computing: the general name for the computational system of this paper. Composed of Φ-computing, λ-computing, and $\mathfrak{M}$-computing; not a third solver with a fixed order of execution, but the boundary keeping two equal computations within the same runtime. **Φ-computing**: the computation that realizes determination as pressure-field compression. The protagonist of this paper. Does not search, does not flee into existing computational methods. **λ-computing**: the computation that inspects Φ's run. Performs only inspection, the origin of learning, and human intervention; does not feed results back into the run (read-only). **$\mathfrak{M}$-computing**: the computation that generates hypotheses. Belongs only to a higher level of abstraction and is not treated in this paper. **The A-field**: the field onto which multiple pressure events are projected. **Pressure event**: the pressure that the local structure of a problem generates, promoting determination. **Pressure bus / collapse cell**: circuit elements that integrate the pressure of each dimension into a single field and collapse it into determination when the pressure difference is decisive. **Quality debt of A**: that determination does not fix because the projection of the pressure field is insufficient. Not compensated by search.

(Note: The proper names of existing solvers and methods mentioned in this paper have been intentionally kept out of the main text to distinguish them from this paper's framework. The details of comparison and correspondence are recorded separately in a technical document. The AC/AJ of this paper are the pressure dimensions on the circuit side, a different concept from the same symbols (distinction of information sources) in the separate line of paradigm treated in the sister paper.)